

📁 OOAD Ch13: 如何尋找 Methods 與 Sequence Diagram

主題重點： CH13 解決的是「class diagram 畫完之後，methods 要怎麼找、要放在哪個 class」的問題。老師的主軸是：先有約 90% 正確的 class diagram，再針對每個 Use Case / scenario 寫 Pseudo Code，逐段分配 responsibility 到適合的 class，讓 methods 自然長出來，最後用 Sequence Diagram 檢查 objects 如何合作完成行為。

👤 Source Materials

- 投影片: [slide 17-1 - how to find methods \(behaviors\) of objects.pdf](#)
- 投影片: [slide 17 - sequence diagram.pdf](#)
- 補充筆記: [ch13note.txt](#)
- 完整逐字稿: [School/OOAD/Transcripts/2026-05-19_CH13_Methods_and_Sequence_Diagram_Transcripts](#)

💡 1. 為什麼 class diagram 之後還要找 methods?

Class Diagram 只完成第一步：找出系統中有哪些 classes、attributes、relationships。真正實作時還要回答：

- 物件有哪些 behaviors / operations?
- 哪些 methods 應該放在哪個 class?
- 一個 use case 的流程要由哪些 objects 合作完成?
- 每個 method 是否真的會被呼叫?

老師提醒：OOAD 要把 behavior 放在正確 class 裡，才有助於降低 maintenance problems。若 logic 放錯位置，未來功能擴充與維護會變困難。

🔧 2. 找 methods 的兩種 approach

2.1 Maximize Approach

適用於設計 generic class library，例如 Java API 或通用 framework。

特色：

- 設計者不知道未來所有 application 會怎麼使用這些 classes。
- 因此傾向盡量提供很多 methods 與不同參數版本。
- 目標是「do as much as you can」。

缺點：

- 若用在有明確 deadline 的 project，容易寫出大量沒人呼叫的 methods。
- 老師稱這種現象為 **C++ newbie syndrome**。
- 典型錯誤：先憑空想像 class 可能需要什麼 methods，全部寫完，最後系統跑起來只用到兩三個。

2.2 Minimize Approach

適用於大多數有明確規格與 deadline 的 project，例如這次 [UML Editor](#)。

特色：

- 只實作規格書需要的 methods。
- 每個 method 都是仔細規劃的。
- 每個 method 會被哪些 classes 呼叫，是事先決定的。
- 寫出來的 method 一定會被 invoke。

核心精神：

不要憑空想像 methods；要從 use case / scenario 推导出必要 behaviors。

🌀 3. OO program 的問題：execution flow 被分散

傳統 C program 常可把一個功能寫在同一個 function 裡；但 OO program 中，一個 feature 往往由多個 classes 的多個 methods 合作完成。

再加上：

- [Inheritance](#)
- [Polymorphism](#)
- method dispatch
- small methods spread across objects

execution flow 會變得直覺。這就是為什麼 UML 需要 [Sequence Diagram](#)：它能視覺化某個 scenario / test case 中 objects 的互動流程。

👤 4. Use Case Based Approach

4.1 前提

開始找 methods 前，需要有：

- 一張約 90% 正確的 class diagram。
- 主要 classes / relationships 已經大致完成。
- class diagram 可以還沒有 methods。

4.2 Algorithm

對每一個 use case / test case：

1. 寫 **pseudo code**

- 先像傳統 C main program 一樣，把完整流程寫下來。
- 此時不要急著分散到 objects。

2. 逐段檢查 **pseudo code**

- 判斷這段工作應該由哪個 class 負責。
- 問：哪個 object 擁有執行這段 behavior 所需的 data / responsibility?

3. 建立 **method**

- 把該段 pseudo code 搬到該 class 下。
- 給 method 一個清楚命名。

4. 評估 **maintainability**

- 這段 code 放在這個 class 是否合理?
- 是否容易維護?
- 是否符合 object responsibility?

5. 重複直到 **pseudo code** 全部分配完

簡化流程：

```
Use Case / Scenario
↓
Pseudo Code
↓
Distribute pseudo code into classes
↓
Create methods
↓
Evaluate maintainability
↓
Draw Sequence Diagram
```

5. 老師 B.1 範例：建立 Association Line

這段在音檔 1 中確認有提到，老師稱為 **scenario B.1**。內容是在 UML Editor 中建立一條 association line。

5.1 User Scenario

概念流程：

1. 使用者 Kevin 點擊 `AssociationLineButton`。
2. Kevin 在 `Canvas` 上的 `(x1, y1)` 按下滑鼠左鍵不放。
3. 系統檢查 `(x1, y1)` 是否落在某個 `BasicObject` 的某個 `Port`。
4. Kevin 拖曳滑鼠到 `(x2, y2)`。
5. Kevin 放開滑鼠。
6. 系統檢查 `(x2, y2)` 是否落在另一個 `BasicObject` 的某個 `Port`。
7. 若 source port 與 target port 都合法，就建立 `AssociationLine`。

5.2 Pseudo Code

```
Kevin clicks AssociationLineButton
AssociationLineButton.actionPerformed()
    Canvas.setMode(AssociationLineMode)

Kevin presses mouse at (x1, y1) on Canvas
Canvas.mousePressed(x1, y1)
    for each object o in Canvas
        if o.isInside(x1, y1)
            o1 = o
            port1 = o.getPort(x1, y1)

Kevin drags mouse to (x2, y2)
Canvas.mouseDragged(x2, y2)
    draw preview line from port1 to current point

Kevin releases mouse at (x2, y2)
Canvas.mouseReleased(x2, y2)
    for each object o in Canvas
        if o.isInside(x2, y2)
            o2 = o
            port2 = o.getPort(x2, y2)

if o1 != null and o2 != null and port1 != null and port2 != null
    new AssociationLine(port1, port2)
```

5.3 Methods 會自然長出來

從 pseudo code 分配後，會得到：

- `AssociationLineButton.actionPerformed()`
- `Canvas.setMode(mode)`
- `Canvas.mousePressed(x, y)`
- `Canvas.mouseDragged(x, y)`
- `Canvas.mouseReleased(x, y)`
- `BasicObject.isInside(x, y): boolean`
- `BasicObject.getPort(x, y): Port`
- `AssociationLine.AssociationLine(port1, port2)`

老師特別提醒：如果把 `Port` 的細節納入，就會發現需要 `getPort(x, y)`，也會自然知道 `BasicObject` 內部應該有 `ports` array / list。

6. Sequence Diagram

6.1 Interaction Diagram

Interaction Diagram 描述一群 objects 如何合作完成某個 behavior。注意：

Sequence Diagram 畫的是 **objects**，不是 classes。

通常用來 capture 單一 use case 的 behavior。

6.2 Sequence Diagram 適合做什麼？

- 顯示 objects 之間的 message / method call 順序。
- 讓分散在多個 classes 的 execution flow 變清楚。
- 描述某個複雜 interaction scenario。
- 特別適合看 concurrent processes。

6.3 基本元素

- **Object / Lifeline**：上方物件與往下的生命線。
- **Message**：物件之間的 method call。
- **Return**：回傳值。
- **Self-call**：物件呼叫自己。
- **Creation**：流程中建立新 object。
- **Deletion**：lifeline 結束以 **x** 表示。
- **Synchronous message**：blocking call，呼叫者等待完成。
- **Asynchronous message**：non-blocking，適合 concurrent / multithreading。

6.4 Frame Options

UML 2.0 後加入 frame 來表示控制結構：

- **alt**：if / else，多分支。
- **opt**：optional，單一 if。
- **loop**：迴圈。
- **par**：parallel 平行處理。
- **region**：critical region，同時間只允許一個 thread。
- **neg**：不合法互動。
- **ref**：引用另一張 sequence diagram。
- **sd**：包住整張 sequence diagram。

老師也提醒：sequence diagram 不擅長展示複雜演算法或大量 loop；若重點是控制流程，activity diagram 或 code 本身可能更適合。

7. Collaboration Diagram

Collaboration Diagram 和 sequence diagram 表達意義相近，但呈現方式不同：

- 不用由上往下的時間軸。
- objects 放在平面空間中。
- message 旁用 1, 2, 3... 編號表示呼叫順序。
- 更容易看出 objects 之間的 links / connection structure。

選擇方式：

- 想強調時間順序：用 Sequence Diagram。
 - 想強調物件連結：用 Collaboration Diagram。
-

8. Centralized vs Distributed Control

Centralized Control

一個大 object 包山包海，負責幾乎所有流程，再去呼叫其他 objects。缺點：

- 主要 object 過大。
- 耦合度高。
- 責任不平均。
- 維護困難。

Distributed Control

職責分配到適合 objects：

```
Order -> OrderLine -> Product / StockItem
```

優點：

- 較符合 OO 精神。
 - 每個 class responsibility 更清楚。
 - 比較容易維護。
-

9. 老師推薦工具

老師示範並推薦：

- websequencediagrams.com

優點：

- 用文字語法描述 sequence diagram。
- 比手動畫圖快很多。
- 輸入類似：

```
Kevin->AssociationLineButton: actionPerformed()  
Kevin->Canvas: mousePressed(x1, y1)  
Canvas->BasicObject: isInside(x1, y1)
```

就能自動畫出 sequence diagram。

對我們來說，也可以用 [PlantUML](#) sequence diagram 達成同樣目的，方便批量產生 PDF。

✅ 本章核心觀念總結

1. Class diagram 是第一步；methods 需要從 use cases 推導。
2. Generic class library 可用 maximize，但一般 project 應採 minimize。
3. 不要寫沒人會呼叫的 methods。
4. Use Case Based Approach: use case → pseudo code → distribute to classes → create methods → evaluate maintainability。
5. Sequence diagram 用來看 objects 之間的互動順序，不是拿來完整描述複雜演算法。
6. Interaction diagram 描述 objects，不是 classes。
7. B.1 範例確認是 UML Editor 建立 association line。
8. `actionPerformed()`、`mousePressed()`、`mouseDragged()`、`mouseReleased()`、`isInside()`、`getPort()` 都是從 B.1 pseudo code 自然推導出的 methods。
9. 好的 OO 設計應偏向 distributed control。

🔗 Related Notes

- [School/OOAD/CH12_UML_Class_Diagram_and_Object_Relationships](#)
- [School/OOAD/TermProject_UML_Editor](#)
- [School/OOAD/2026-05-19_UML_Editor_Teacher_Analysis](#)
- [School/OOAD/CH13_UML_Editor_Assignment_Methods_and_Sequence_Plan](#)